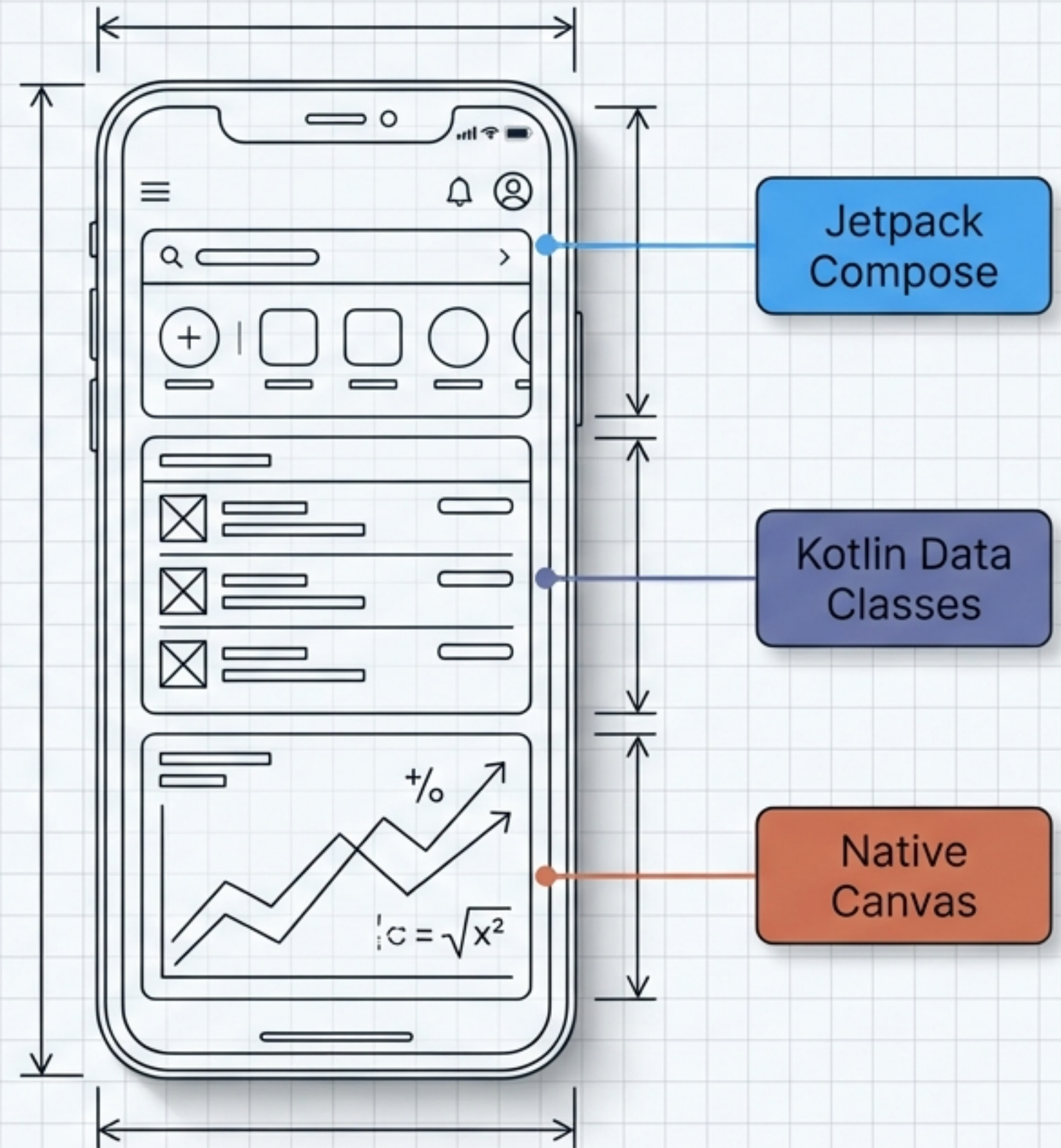


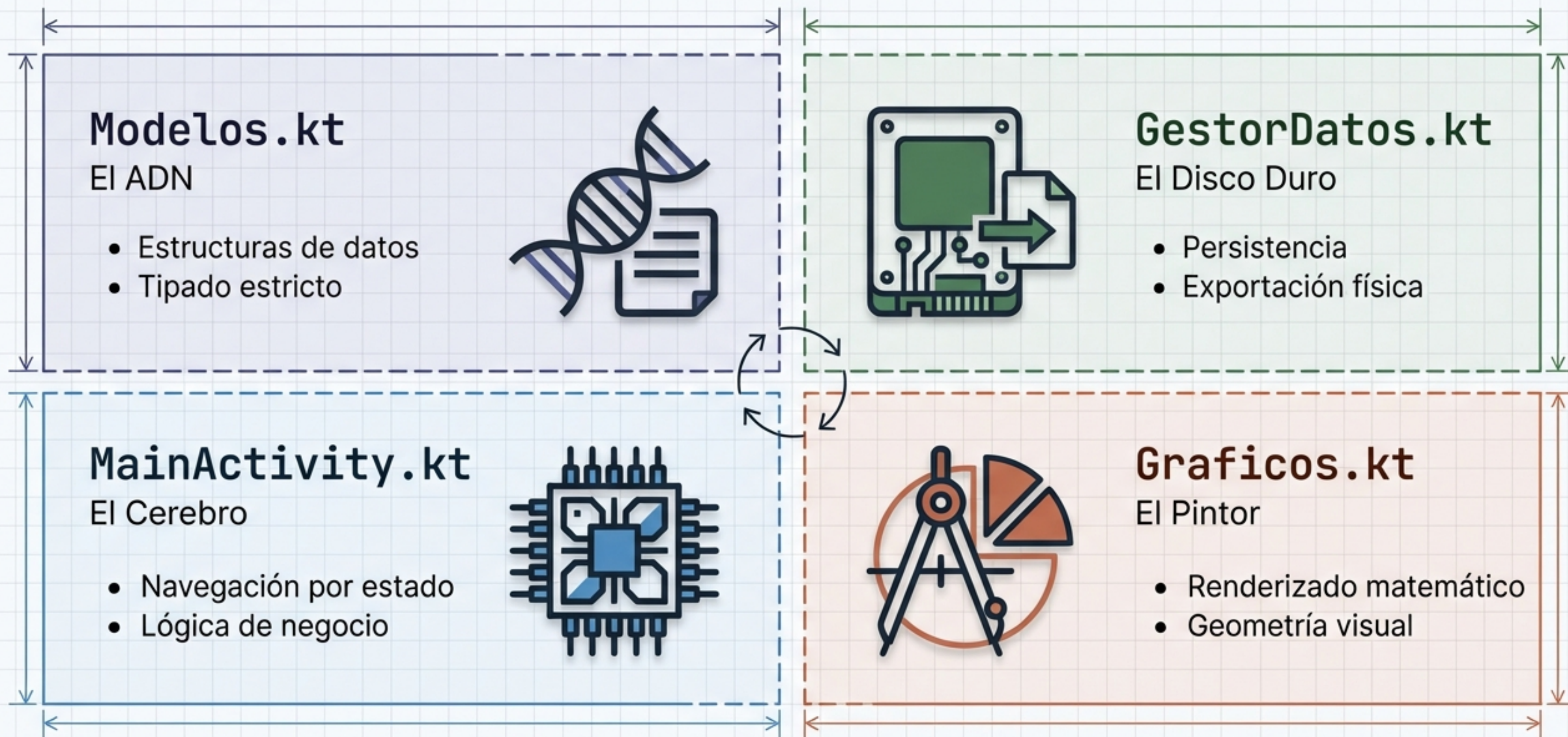
Deconstruyendo MyBudget

Arquitectura, Kotlin y Jetpack Compose para Desarrolladores Android.

Anatomía de una app financiera moderna.

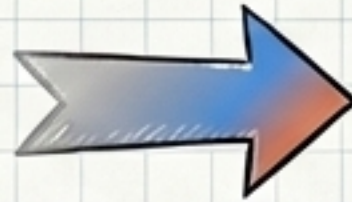


El Mapa Arquitectónico



Cambio de Paradigma

Tradicional



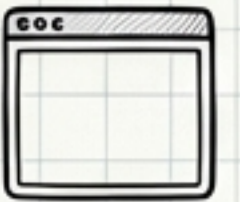
Moderno

Navegación

Intents y múltiples Activities



Single Page App (Estado Reactivo)



Interfaz UI

Archivos XML y findViewById



Jetpack Compose (UI Declarativa)



Gráficos

Librerías externas pesadas



Canvas Nativo (Matemáticas directas)



Datos

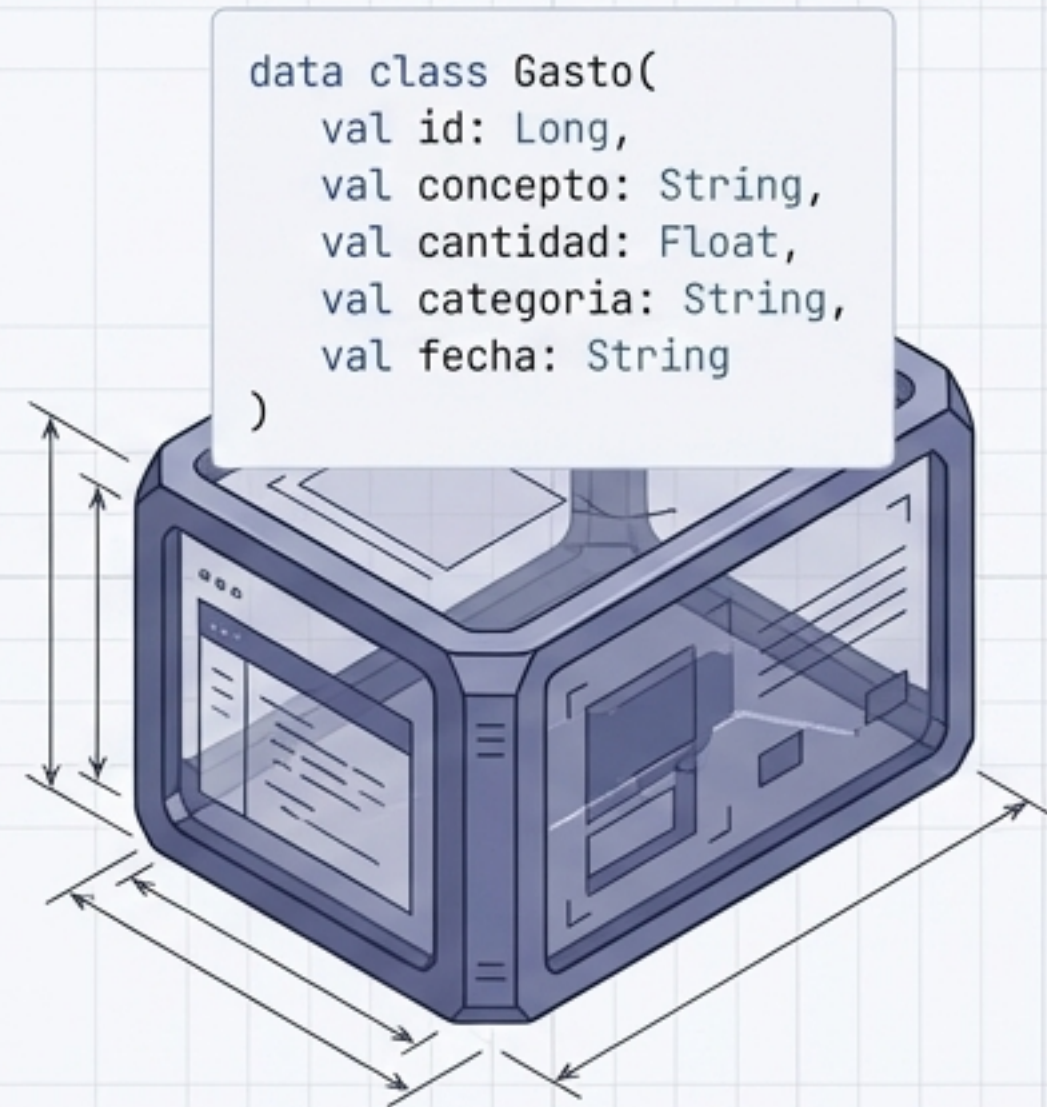
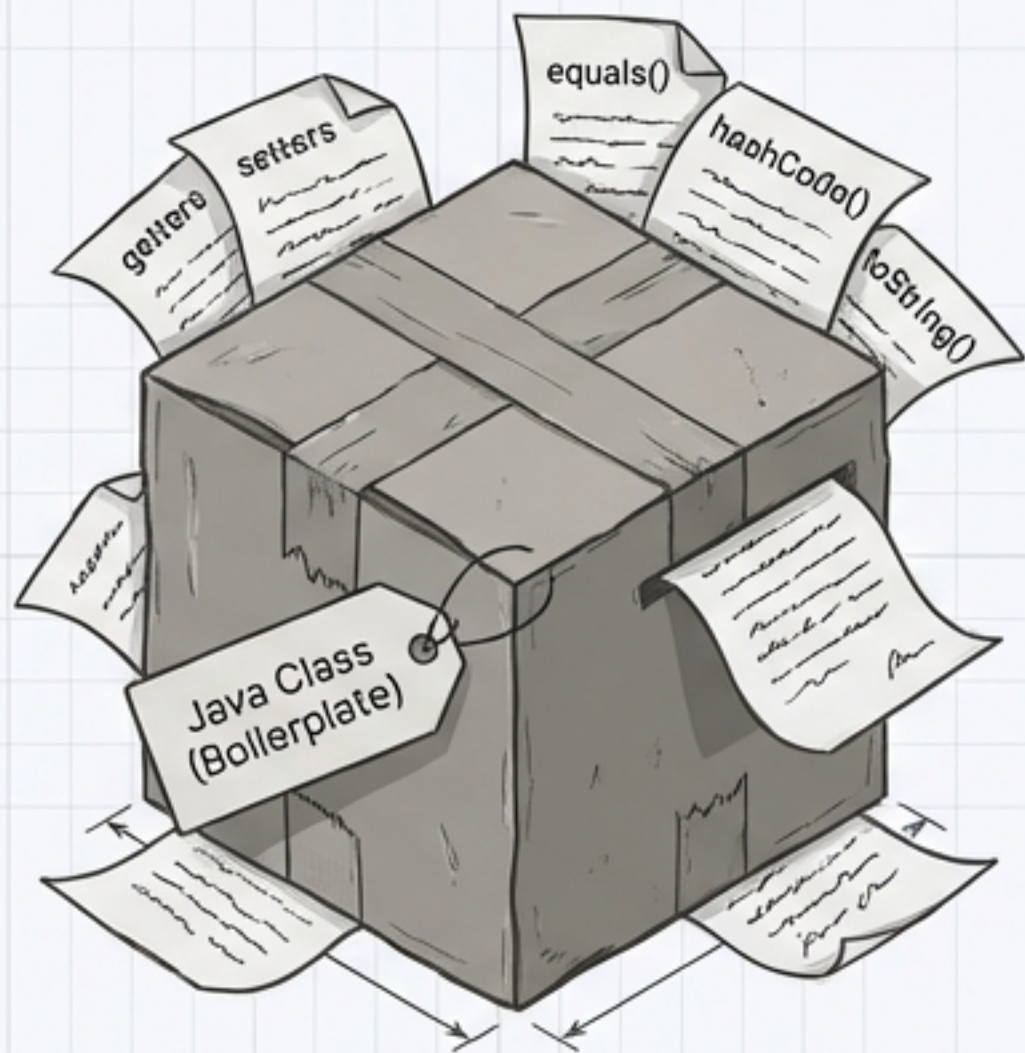
Clases Java con boilerplate



Data Classes de Kotlin (Código cero)



Pilar 1: Datos Inteligentes (Modelos.kt)



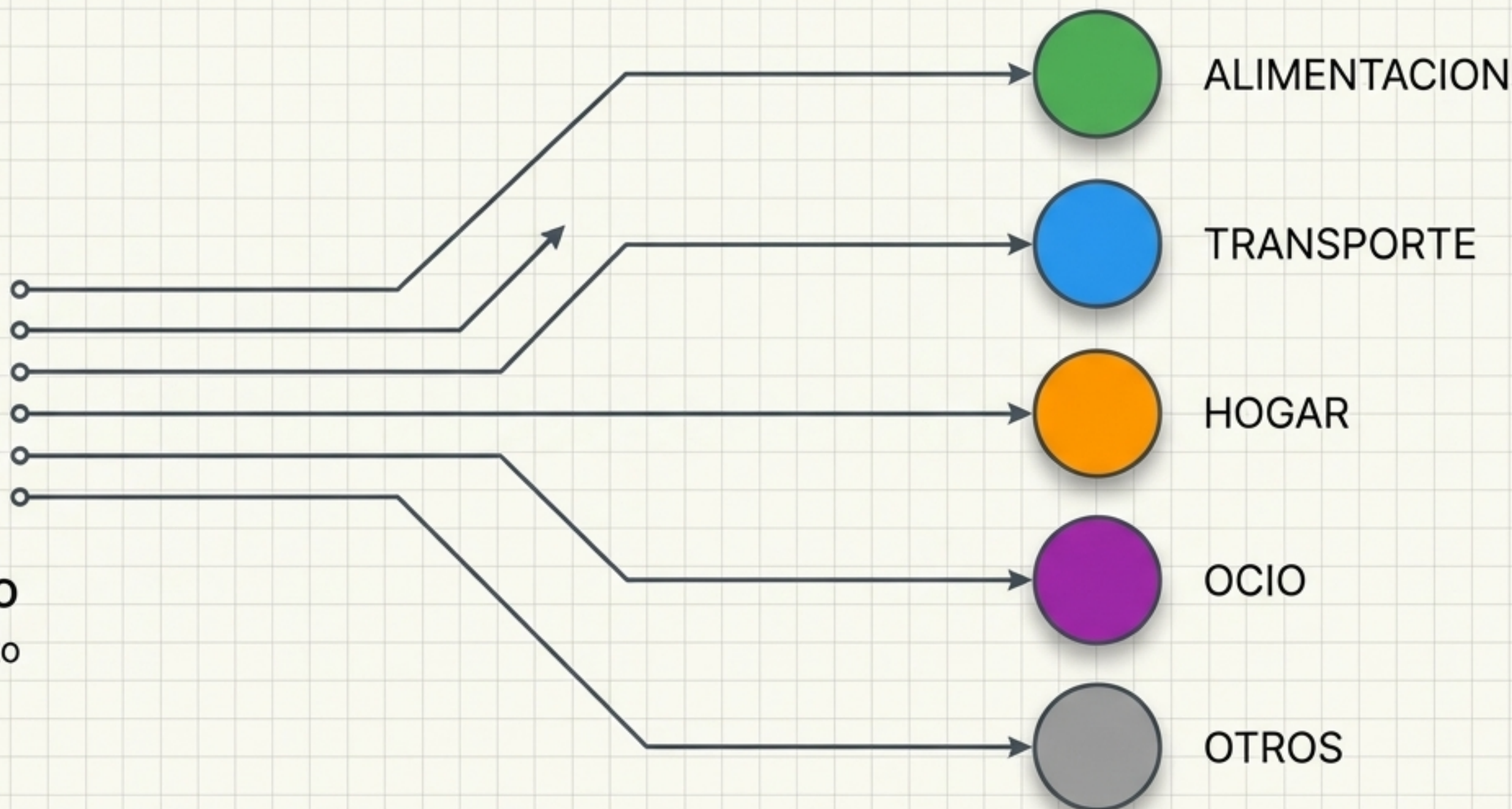
La magia del data class: Kotlin genera automáticamente los métodos ocultos (toString, equals, hashCode) sin escribir una sola línea extra. Empaqueta información transaccional en una única entidad inmutable.

Pilar 1: Tipado Estricto para UI (Modelos.kt)



Tipado Estricto

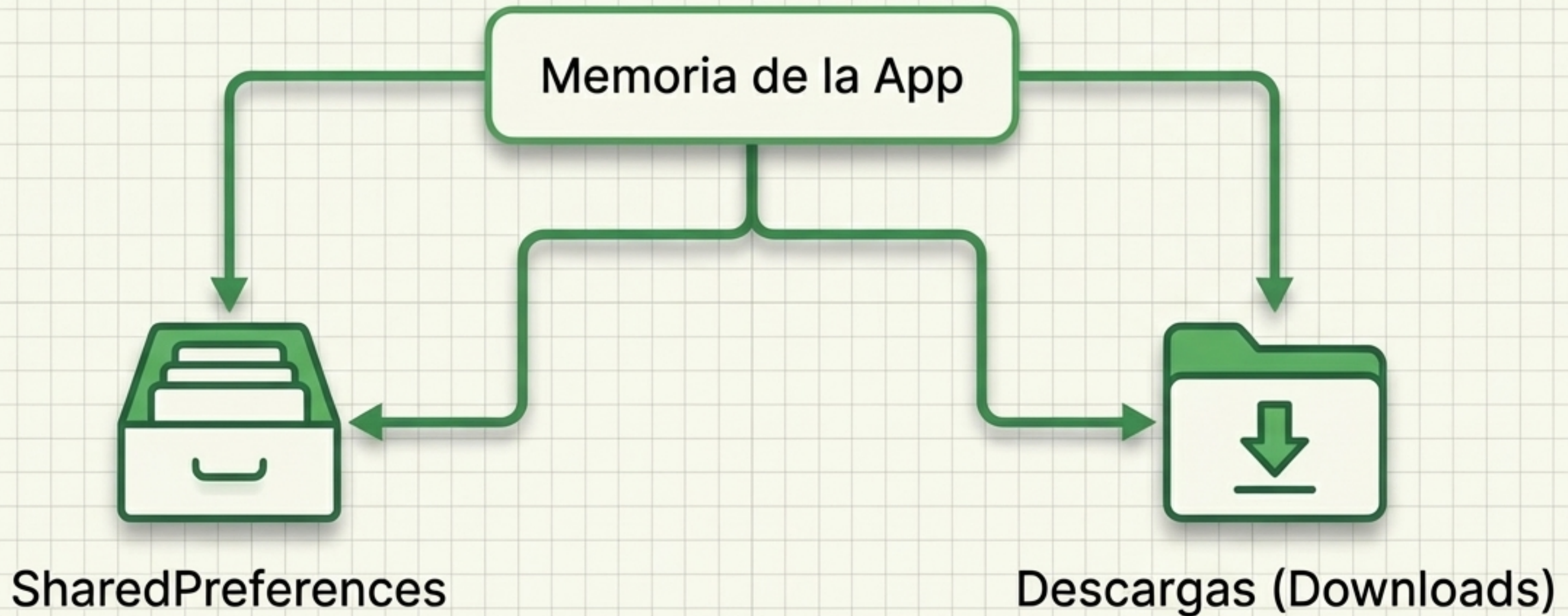
Enum: CategoriaGasto



El Enum no solo previene errores tipográficos. Al incrustar el objeto Color directamente, creamos un puente directo entre los datos puros y el renderizado visual.

```
fun obtenerColorPorNombre(nombre: String): Color
```


Pilar 2: El Disco Duro y Android Nativo (GestorDatos.kt)



Almacenamiento ligero. Evitamos arquitecturas complejas como Room/SQL para mantener el código accesible y sin fricción.

Rutas públicas. Al exportar el PDF, escribimos directamente en el directorio Environment público para fácil acceso.

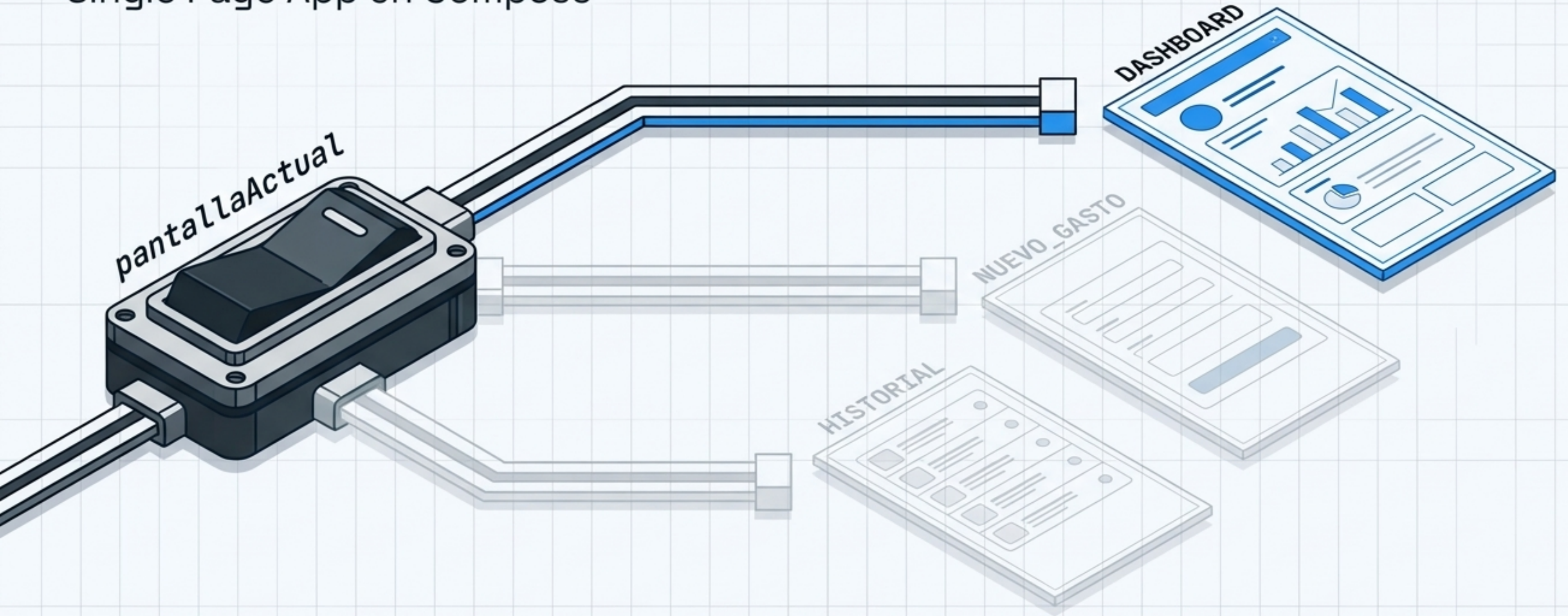
Pilar 2: El 'Hack' del Visor Externo (GestorDatos.kt)



Prioridad Educativa: Compartir archivos físicos en Android moderno normalmente exige la configuración compleja de un FileProvider. Para priorizar el aprendizaje del motor de renderizado de PDFs sobre el boilerplate de seguridad, el código utiliza una omisión del StrictMode, forzando la apertura directa en un visor externo.

Pilar 3: Navegación por Estado (MainActivity.kt)

Single Page App en Compose



Adiós a `startActivity()`. En la arquitectura moderna, no "saltamos" entre pantallas. Simplemente cambiamos una variable de estado. Compose reacciona instantáneamente y redibuja la interfaz completa. La pantalla muta, no se desplaza.

Pilar 3: Matemáticas Funcionales (MainActivity.kt)

Total

```
gastos.map { }.sum()
```

Extrae las cantidades y las suma en una sola línea.

Días Activos

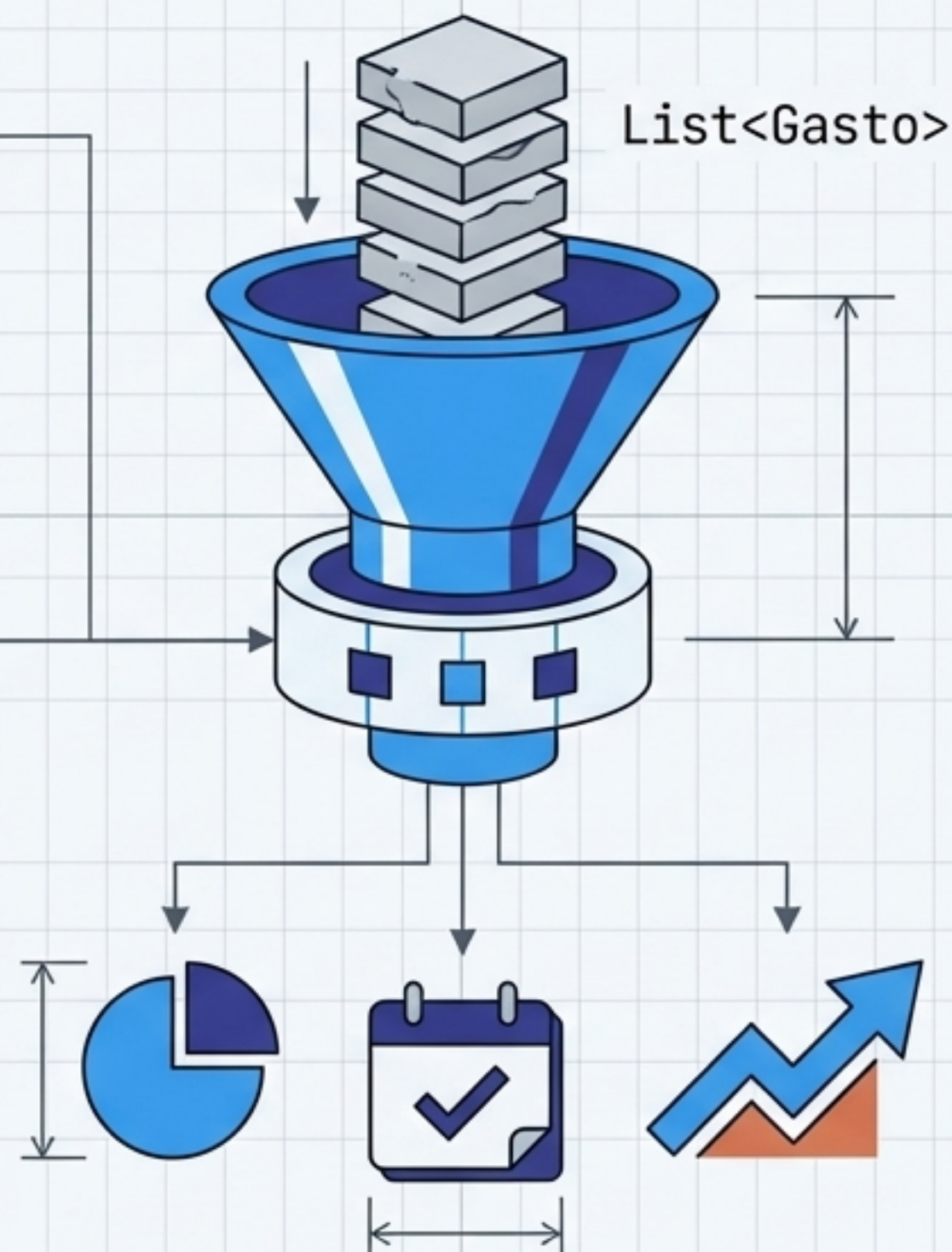
```
gastos.map { it.fecha }.distinct().size
```

Filtra fechas únicas para saber exactamente en cuántos días se ha gastado dinero.

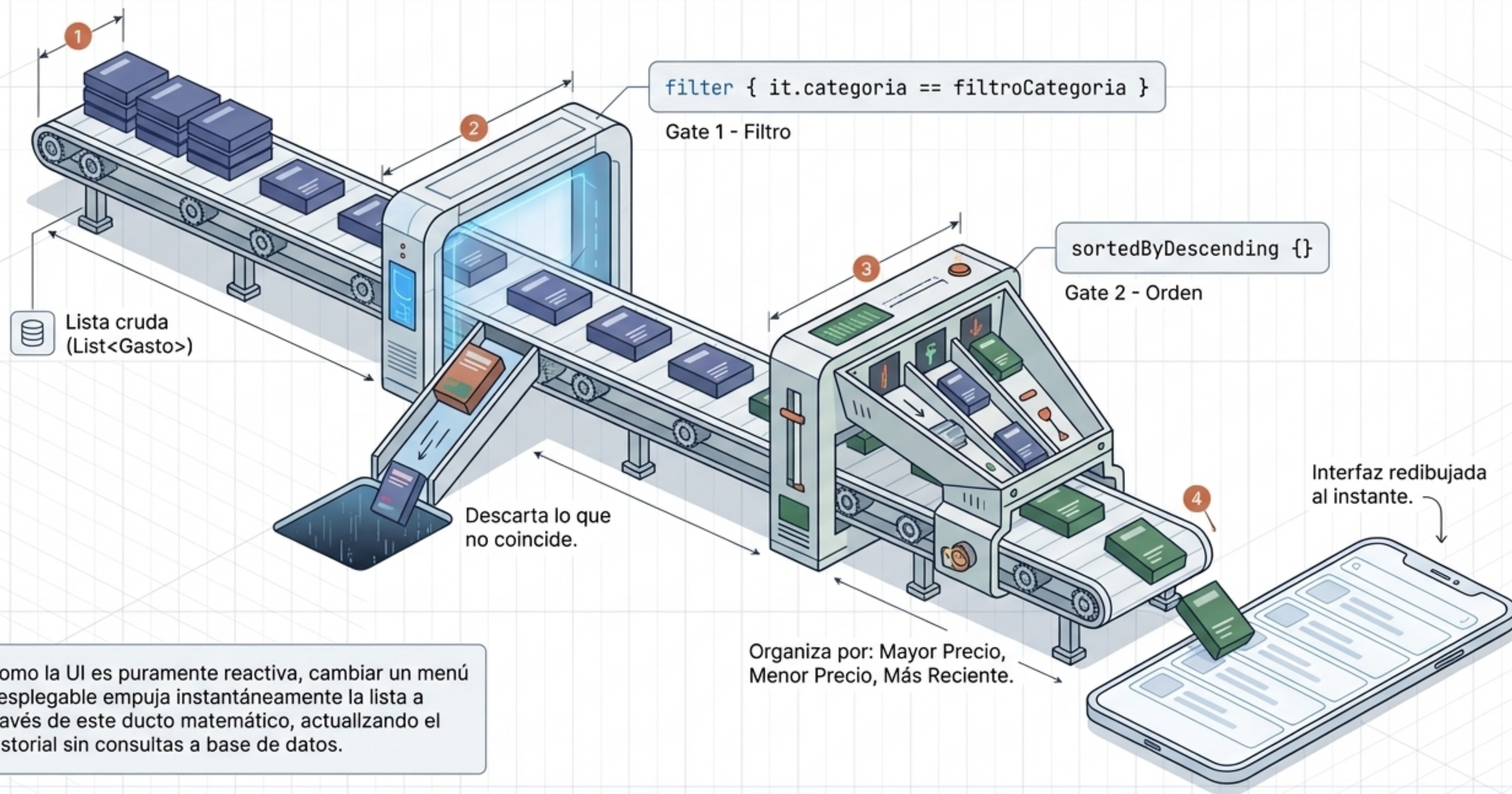
Predicción Mensual

```
(Total / Días Activos) * 30
```

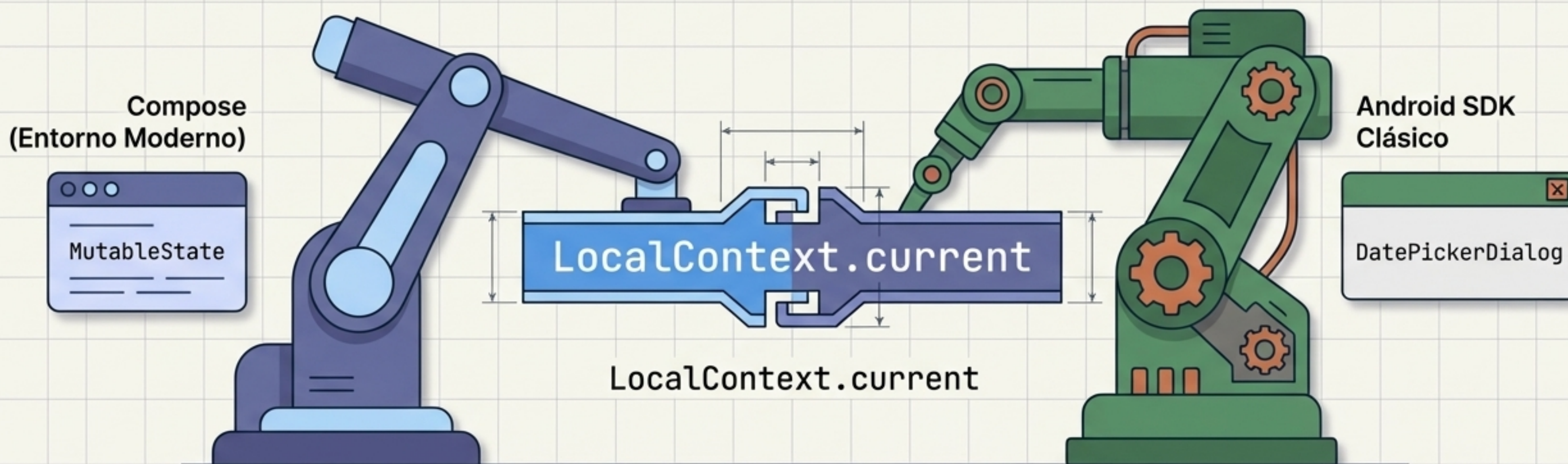
Calcula el flujo de caja estimado basado en el promedio diario.



Pilar 3: El Pipeline de Filtrado (MainActivity.kt)

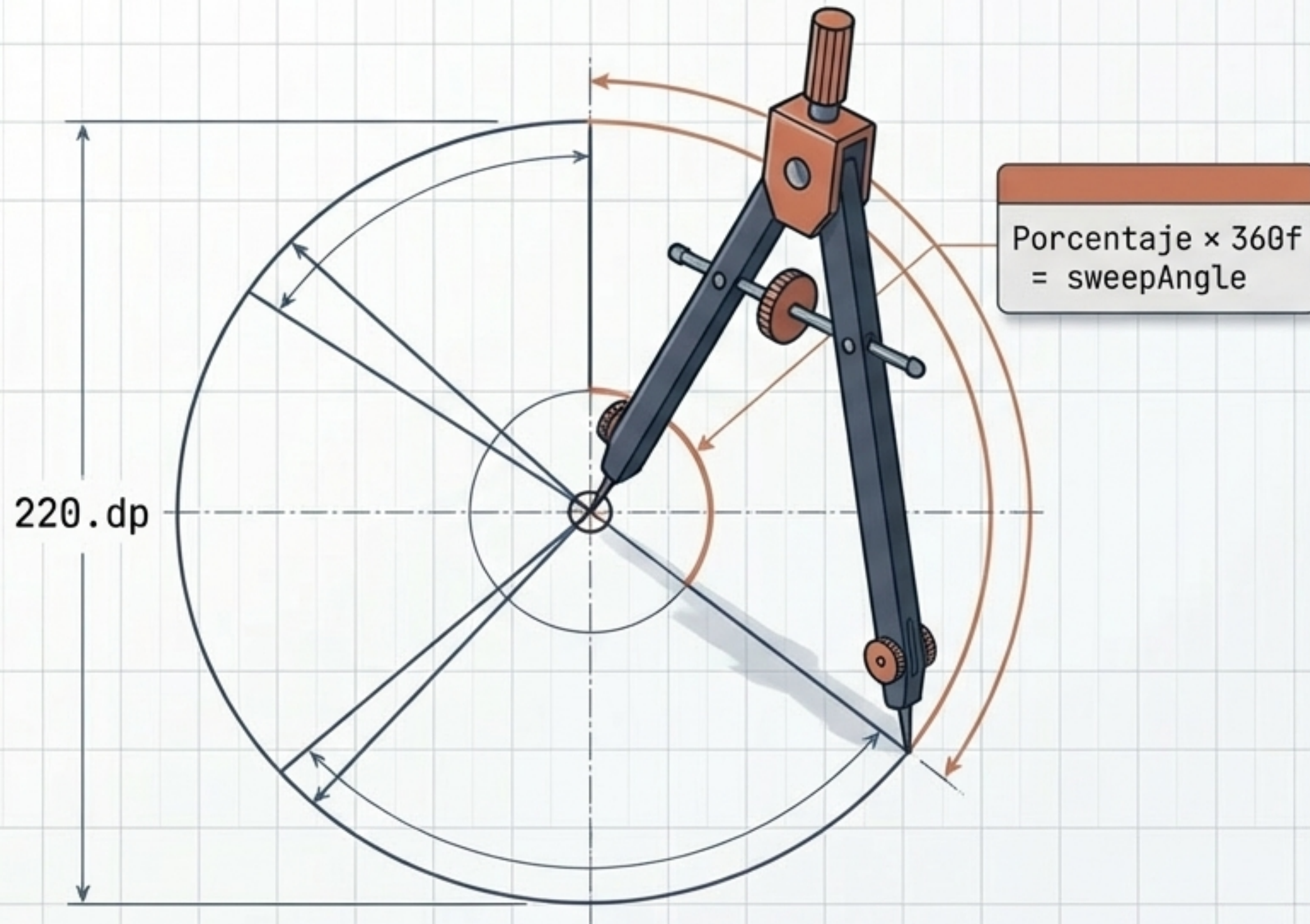


Pilar 3: Interoperabilidad con el Pasado (MainActivity.kt)



¿Cómo usamos un calendario clásico dentro de un entorno moderno? Usamos `LocalContext.current` para invocar el antiguo `DatePickerDialog`. Al seleccionar la fecha, capturamos el resultado nativo y lo inyectamos de vuelta en un estado de Compose, uniendo ambos mundos a la perfección.

Pilar 4: Renderizado Custom (Graficos.kt)

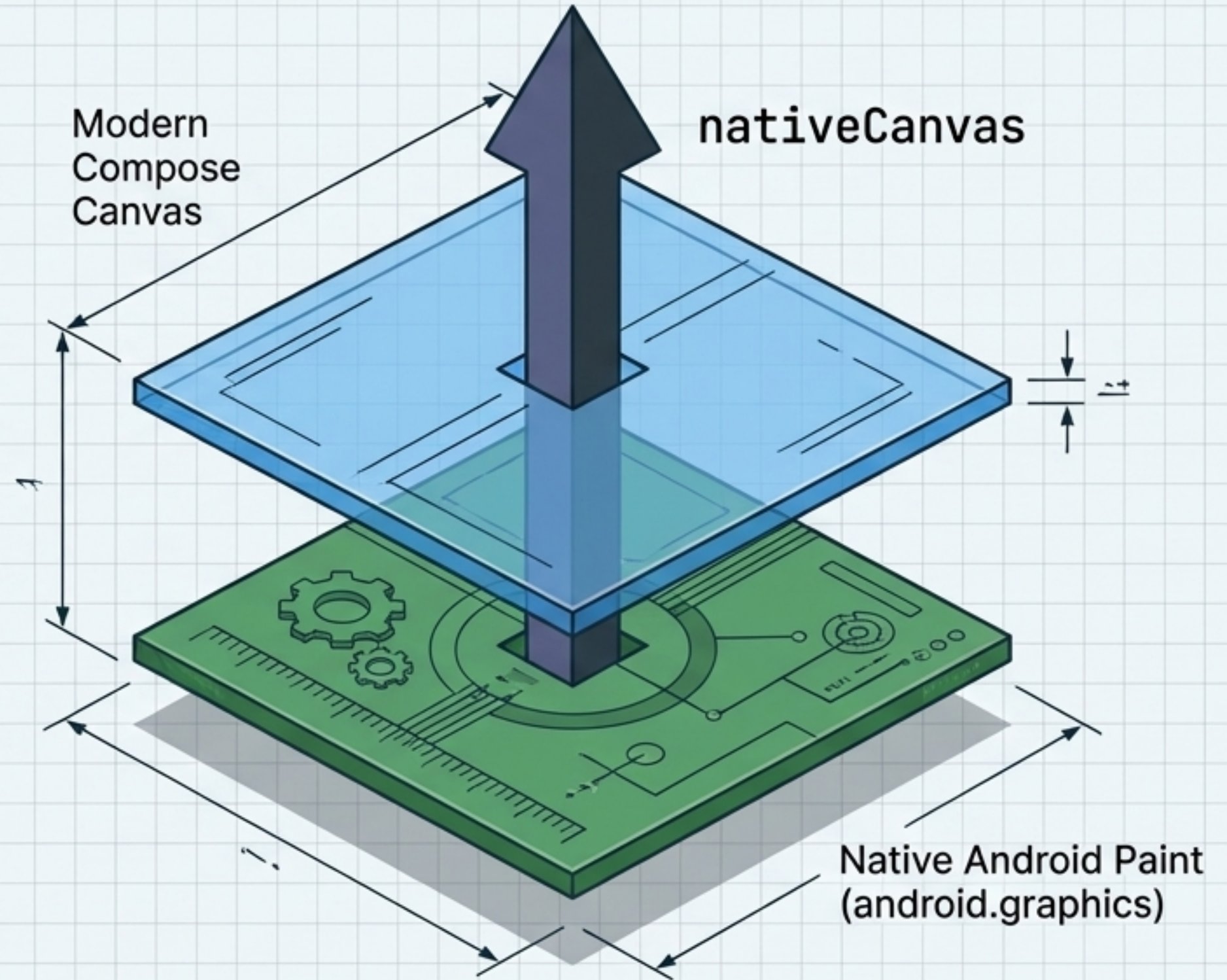


- 1 Transformación:** Los datos agrupados (`sumasPorCategoria`) se convierten en porcentajes.
- 2 Cálculo de Ángulos:** Ese porcentaje se multiplica por 360f para determinar los grados exactos del trozo.
- 3 Renderizado:** Jetpack Compose Canvas dibuja matemáticamente cada arco de forma secuencial.

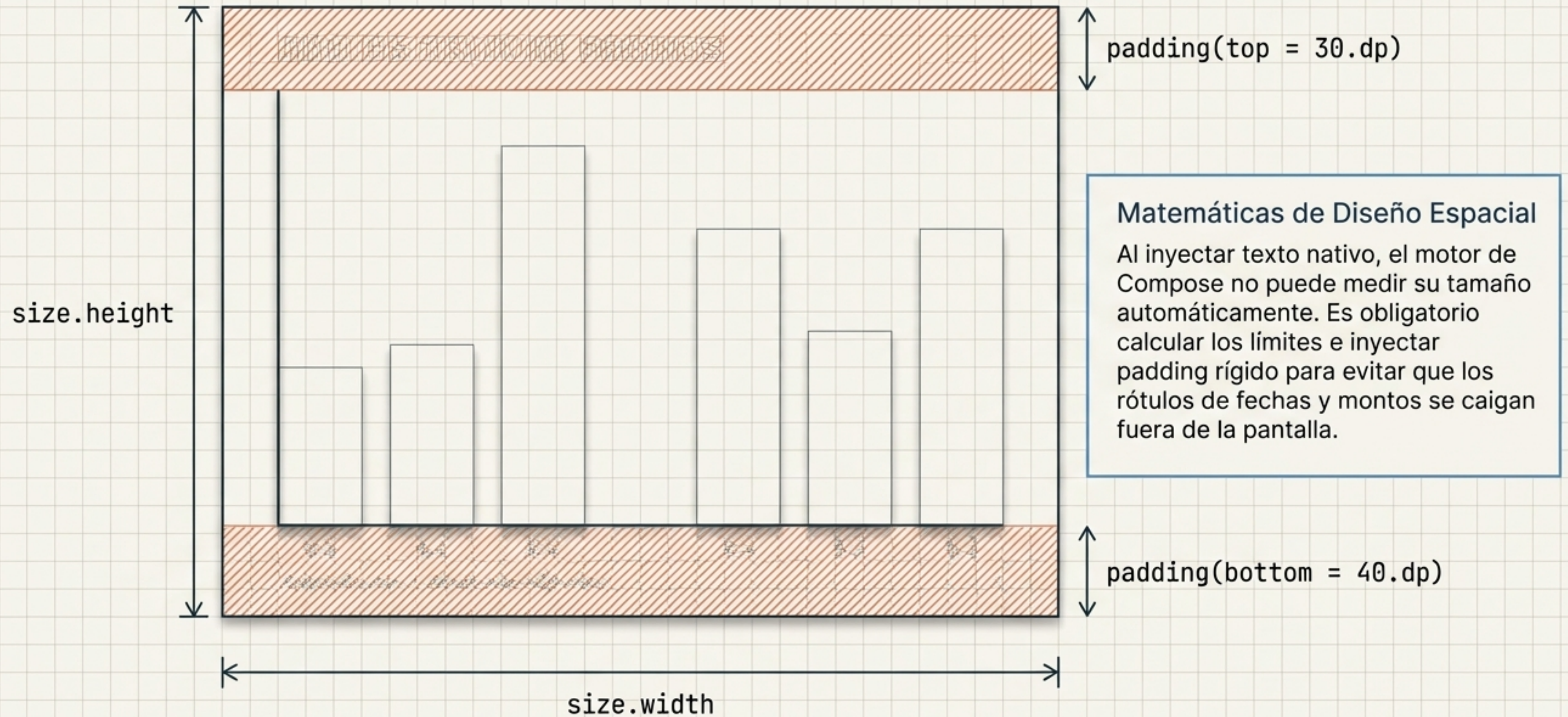
Pilar 4: La Vía de Escape Nativa (Graficos.kt)

El Problema: El Canvas de Jetpack Compose tiene limitaciones al dibujar texto complejo y tipografías nativas.

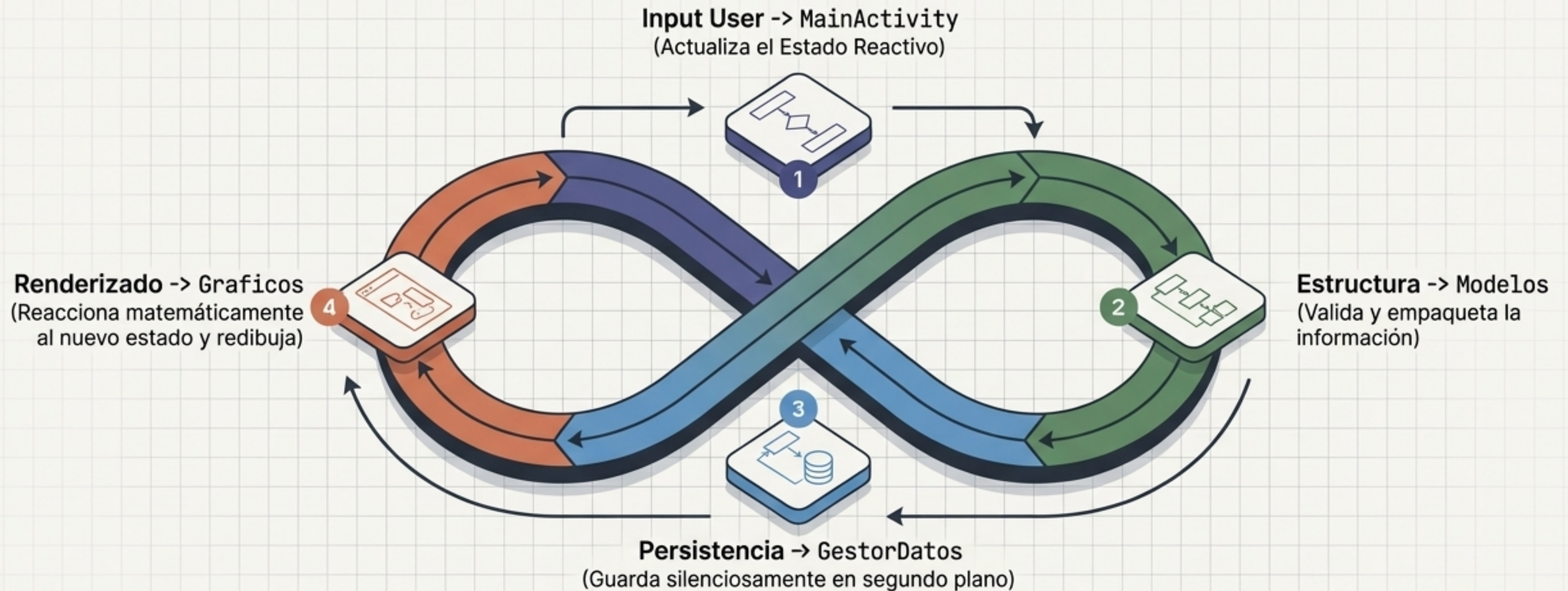
La Solución: El código utiliza la función `nativeCanvas` para ‘escapar’ temporalmente del entorno moderno. Esto permite acceder a la clase clásica `android.graphics.Paint`, habilitando rótulos precisos sobre las figuras geométricas.



Construyendo el Gráfico de Barras (Graficos.kt)



El Flujo Completo (The Synthesis Arc)



En esta arquitectura, los datos son la única fuente de verdad, el estado es el director de orquesta, y la interfaz gráfica es simplemente un reflejo matemático instantáneo de ese estado.